Real Data Engineer Interview Q&A

(Spark, PySpark, Databricks, Delta Lake)

1 Spark Job Runs Slowly — How Do You Debug It?

What they asked me:

"Our Spark job is taking 3× longer than usual. How will you identify the bottleneck?"

What I said:

I open the **Spark UI** and check:

- **Stages tab** → look for stages with long runtimes or large shuffle reads/writes.
- **Tasks tab** → identify **straggler tasks**, often caused by skew.
- **Executors tab** → check memory spills, GC time, executor failures.

Then I verify:

- If a wide transformation triggered a huge shuffle (visible as ShuffleExchange in the DAG).
- Whether incorrect partitioning (too few partitions) reduces parallelism.
- Whether a join can be turned into a **broadcast join** to eliminate shuffle.
- If data is uneven → use **salting** or **AQE skew optimization**.

Tips:

Always start with:

1. Long stage? → shuffle problem
2. Long task? → skew
3. Memory spill? → increase partitions or executor memory

---

2 Difference Between Repartition and Coalesce — Real Use Case

What they asked me:

"Which one will you use when writing Delta data to S3 in production?"

What I said:

- **coalesce()** reduces partitions *without shuffle*, best for final writes to avoid small files.

- **repartition()** adds or redistributes partitions *with shuffle*, best when preparing for joins.

Example:
If a job created **thousands of micro-partitions** after a shuffle, I do:

```
final_df = df.coalesce(20)
```

This reduces files and speeds up downstream reads.

Tips:

Use **coalesce for fewer files**, **repartition for parallelism**.

---

3 Explain a Time You Handled Skewed Data

What they asked me:

"How did you fix data skew in a real pipeline?"

What I said:

In a customer-transactions pipeline, one customer had **millions** of rows, others had few. GroupBy caused one partition to do all the work → job stuck at 95%.

I applied:

- **Salting:** add a random prefix to keys.
- Increase shuffle partitions.
- Enable **AQE** (`spark.sql.adaptive.enabled = true`).

This split the heavy partition into multiple sub-tasks.

Tips:

If one key appears too often → **salting or broadcast join** is the safest fix.

---

4 Why Is Delta Lake Better Than Parquet? Explain With Scenario

What they asked me:

"How is Delta Lake superior to plain Parquet in a real pipeline?"

What I said:

In ingest pipelines:

- Parquet cannot handle **updates/deletes**, but **Delta supports MERGE INTO**.
- Parquet has no **ACID**, so concurrent writes may corrupt data.
- Delta **transaction log** tracks versions → enables time travel.
- Auto-optimizations like **ZORDER** and **OPTIMIZE** improve read performance.

Example scenario:
CDC updates from Kafka → Delta MERGE handles row-level updates without rewriting entire tables.

Tips:

Always highlight:
ACID + Time Travel + MERGE INTO + Schema Enforcement → **real enterprise value**.

---

5 How Does the Catalyst Optimizer Improve Performance?

What they asked me:

"Explain Catalyst in simple, interview-friendly terms."

What I said:

Catalyst performs:

- **Logical optimization** → predicate pushdown, removing unnecessary columns.
- **Physical planning** → automatically deciding broadcast vs shuffle join.
- **Code generation (Tungsten)** → compiles operations to JVM bytecode.

Result:
The query I write in PySpark may not be how Spark executes it — Spark generates the **best possible plan** internally.

Tips:

Catalyst = **Brain**, Tungsten = **Muscle**.

---

6 How Do You Handle Small Files in Delta Lake?

What they asked me:

"Our Silver layer is generating thousands of small Delta files. What is your approach?"

What I said:

I use:

- `OPTIMIZE table_name` → compacts small files to larger ones.
- `ZORDER BY (important_column)` → improves data skipping.
- Autoloader with **trigger once** to reduce micro-batches.
- Tune batch size to generate fewer files.

Tips:

Small files = slow queries + high metadata overhead → always compact before Gold layer.

---

7 Explain MERGE INTO in a Real CDC Pipeline

What they asked me:

"How do you load CDC data from Kafka into Delta?"

What I said:

I use **MERGE INTO**:

- If record exists → UPDATE
- Else → INSERT

This handles inserts, updates, and deletes.

Example:

```
MERGE INTO target t
USING source s
ON t.id = s.id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
```

Tips:

MERGE + Delta = perfect for slowly changing or transactional data.

---

8 How Would You Optimize a Spark Join?

What they asked me:

"Your job has a huge shuffle during a join — how do you fix it?"

What I said:

I check:

1. Is one table small (< 10 GB)? → **broadcast join**.
2. Are tables partitioned on join key?
3. Use **bucketing** for repeated joins.
4. Enable **AQE** to switch strategy at runtime.
5. Apply **filter early** and **prune columns**.

Tips:

If shuffle is huge → **broadcast first**, then check skew.

---

9 Explain Checkpointing in Streaming Pipelines

What they asked me:

"How does your Structured Streaming pipeline recover on failure?"

What I said:

Using:

```
.option("checkpointLocation", "path")
```

Spark stores:

- Offsets
- State
- Watermark info
- Progress logs

When a failure occurs, Spark resumes **exactly where it left off**.

Tips:

No checkpoint → **no fault tolerance** in streaming.

---

10 How Do You Decide Partitioning Strategy in Delta?

What they asked me:

"How do you choose a partition column?"

What I said:

I check:

- Query patterns (WHERE, GROUP BY).
- Cardinality of column.
- Avoid high-cardinality (customer_id).
- Prefer time-based (date, month) for incremental loads.

Tips:

Best partitions:
✓ date
✓ region
Avoid:
✕ id
✕ random UUID

---

11 PySpark UDF vs Pandas UDF vs Native Functions

What they asked me:

"Why should we avoid Python UDFs?"

What I said:

- Python UDFs → break Catalyst optimizer + slow due to JVM–Python transition.
- Pandas UDFs → use Apache Arrow, vectorized, much faster.
- Native Spark SQL functions → **fastest** and fully optimized.

I always start with:
→ Native functions
→ If impossible, Pandas UDF
→ Avoid normal UDFs

Tips:

If interviewer asks:
"Which one should you always prefer?"
Answer: **Spark SQL built-ins.**

---

12 How Does Autoloader Improve File Ingestion?

What they asked me:

"Why did you choose Autoloader instead of a normal readStream()?"

What I said:

Autoloader (cloudFiles):

- Handles **billions** of files.
- Uses **file notification APIs** instead of directory listing.
- Prevents duplicate processing.
- Automatically infers schema + supports evolution.

It reduces operational load and improves scalability.

Tips:

For large-scale file ingestion in Bronze → **always prefer Autoloader**.